

ParkStub: A Full-Stack Parking Reservation and Slot Management System

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

Shreenitha.D – 2520030504

Srihitha.A – 2520030566

Under the Guidance of

Dr.S.Spandhana

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
Declaration

The Project Report entitled "ParkStub: A Full-Stack Parking Reservation and Slot Management System" is a record of bonafide work of Shreenitha.D – 2520030504, Srihitha.A – 2520030566 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The work presented in this report describes the design and implementation of a full-stack parking reservation and slot management application.

The project demonstrates the integration of a React frontend, Node.js and Express backend, and MySQL database. The report describes the system requirements, architecture, design, implementation, features, testing, deployment approach, limitations and future enhancements.

Shreenitha.D – 2520030504

Srihitha.A – 2520030566

K L (Deemed to be) University

CERTIFICATE

This is to certify that the mini project based report entitled "ParkStub: A Full-Stack Parking Reservation and Slot Management System" is a bonafide work done and submitted by Shreenitha.D – 2520030504 and Srihitha.A – 2520030566 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Computer Science Engineering.

The project was developed under the guidance of Dr.S.Spandhana. The report records the design, development, testing and integration of the ParkStub application.

Signature of the Guide Signature of the Course Coordinator Signature of the HOD

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr.S.Spandhana for the guidance, encouragement and support provided during the development of the ParkStub project. The guidance helped us understand the requirements, organize the implementation and improve the final application.

We also thank the faculty and department for providing the academic environment and resources required to complete this work. We are grateful to the teaching and non-teaching staff members who supported us directly or indirectly during the project.

Finally, we sincerely thank our parents, friends and classmates for their encouragement and cooperation throughout the development of the project.

Shreenitha.D – 2520030504

Srihitha.A – 2520030566

Abstract

ParkStub is a full-stack web application developed to simplify parking discovery and reservation. The system allows users to register or log in, browse different parking places, view place-specific parking slots, select a date and available slot, complete a payment flow and receive booking confirmation.

The application is implemented using React for the frontend, Node.js and Express for the backend, and MySQL for persistent data storage. REST APIs connect the frontend with the backend, while the backend manages validation, booking operations and database communication.

A key feature of ParkStub is realistic slot management. Different parking places can have different combinations of available, occupied and reserved slots. This prevents the interface from displaying the same availability pattern for every location and provides a clearer representation of parking conditions.

The system also provides My Bookings and cancellation/refund functionality according to the implemented workflow. Overall, the project demonstrates the practical integration of user interface design, server-side APIs, relational database management and end-to-end testing in a real-world parking reservation scenario.

Index

Index – Continued

Acknowledgement – 4

Abstract – 6

Chapter 1 Introduction – 5 to 15

Chapter 2 System Requirements – 16 to 21

Chapter 3 Technology Stack – 22 to 32

Chapter 4 System Architecture – 33 to 36

Chapter 5 Design – 37

Chapter 6 Implementation – 38 to 47

Chapter 7 Features – 48 to 55

Chapter 8 Testing – 56 to 64

Chapter 9 Deployment – 65 to 73

Chapter 10 Challenges & Limitations – 74 to 82

Chapter 11 Future Enhancements – 83 to 88

Chapter 12 Conclusion – 89 to 97

Chapter 13 References – 98 to 99

Chapter 14 Appendices – 100 to 105

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

ParkStub is a web-based parking reservation and slot management system designed to simplify the process of finding, selecting and reserving parking spaces. Traditional parking often depends on physical signs, manual coordination and uncertain availability. ParkStub brings the main activities into one digital interface so that a user can view parking places, inspect slot status and complete a reservation before arriving.

The project follows a full-stack architecture. The React frontend provides the user interface, while a Node.js and Express backend exposes REST APIs for authentication, parking-place data, slot management, bookings, payments and related operations. MySQL stores structured information such as users, parking locations, slots, reservations and booking records. This separation keeps the presentation, business logic and data storage responsibilities clear.

A major requirement of ParkStub is to represent realistic slot conditions. Each parking place can have its own set of available, occupied and reserved slots. This allows users to understand the current state of a parking area rather than seeing every location with an identical availability pattern.

1.1 Background of the Project

The project was developed as a practical application of frontend development, backend API design and database management. It demonstrates how a user action in a browser can travel through a REST API to a database and return an updated result to the interface. The system therefore connects user experience with server-side processing and persistent storage.

The design also considers the complete booking journey. A user can register or log in, select a parking place, choose a date and available slot, proceed through the payment flow, receive booking confirmation and later review or cancel a booking according to the implemented rules. This creates an end-to-end workflow instead of a collection of unrelated pages.

1.2 Problem Statement

1.2 Problem Statement

Finding reliable parking can be difficult when drivers do not know which locations have free spaces. A user may reach a parking place only to discover that spaces are occupied, reserved or unavailable. Manual processes also make it difficult to maintain consistent booking records and prevent two users from selecting the same space.

The problem addressed by ParkStub is the lack of a single, convenient system that combines parking-place discovery, slot availability, reservation, payment and booking history. The system should show slot status clearly and allow a user to select only an available slot for a chosen date and time.

A second problem is data consistency. The frontend should not independently decide the final booking state. Instead, the backend validates requests, communicates with MySQL and updates the booking information. This creates a controlled flow between the interface and the database.

1.2 Problem Statement

ParkStub therefore treats a parking slot as a managed resource. Each place contains multiple slots, and each slot can have a status such as available, occupied or reserved. When a reservation is created, the corresponding booking information is stored and the interface can reflect the changed state.

The solution is intended to reduce uncertainty for users, improve organization for parking operators and demonstrate a scalable software structure for future parking-related services.

1.3 Scope of the Project

1.3 Scope of the Project

The scope of ParkStub covers the main activities required for online parking reservation. Users can create an account, authenticate, browse parking places, open a location, view slot information, select a date and choose an available slot. The system then processes the booking request and presents confirmation details.

The frontend scope includes responsive React pages for the home view, parking-place listings, place details, slot selection, booking, payment, confirmation and My Bookings. The backend scope includes REST endpoints for users, parking places, slots, bookings, payments and cancellation-related operations.

The database scope is focused on structured relational storage. MySQL is used to keep user information, parking-place records, slot records and booking/payment-related data. Relationships between these records help maintain consistency and make it possible to retrieve a user's booking history.

1.3 Scope of the Project

The current project is primarily a local full-stack implementation. It demonstrates the complete application flow rather than claiming production deployment across a large network of parking operators. Features such as live sensor-based occupancy, GPS navigation, automatic license-plate recognition and large-scale operator dashboards can be considered future extensions.

The project can also be extended with real payment gateways, notifications, map services and real-time updates. The current architecture is suitable for such additions because the frontend and backend communicate through defined APIs.

1.4 Objectives of the Project

1.4 Objectives of the Project

The first objective is to develop a user-friendly parking reservation interface that reduces the effort required to find and reserve a parking slot. The interface should present parking places clearly and make the next action understandable to the user.

The second objective is to provide place-specific slot availability. Different parking places should contain different combinations of available, occupied and reserved spaces so that the application represents realistic conditions.

The third objective is to connect the React frontend with a Node.js and Express backend through REST APIs. Requests such as registration, login, place retrieval, slot selection and booking should be processed by the server instead of being handled only in the browser.

The fourth objective is to persist important information in MySQL. This includes users, parking locations, slots and booking records, allowing information to remain available across sessions.

1.4 Objectives of the Project

The fifth objective is to implement a complete reservation journey including date selection, slot selection, payment flow and booking confirmation. The sixth objective is to provide a My Bookings area where users can view their previous reservations and use the supported cancellation or refund flow.

Together, these objectives demonstrate the practical integration of frontend development, backend services, database management and application testing.

1.5 Importance of the Application

1.5 Importance of the Application

Parking is a common real-world problem in busy areas. A digital reservation system can make parking information easier to access by presenting location and slot information before the driver reaches the parking area. ParkStub demonstrates this idea through a simple browser-based workflow.

The application is important from a software engineering perspective because it combines several common full-stack requirements in one system: authentication, CRUD-style data access, REST APIs, relational database operations, state changes, booking validation and user-facing confirmation.

The slot-status model is particularly useful because it distinguishes between spaces that can be selected and spaces that should not be selected. Available slots can be offered for booking, while occupied and reserved slots are displayed as unavailable.

1.5 Importance of the Application

For users, the application provides visibility and a structured booking process. For the system, it creates a centralized record of reservations. For students and developers, it provides a practical example of how a React interface, Express server and MySQL database can work together in an end-to-end application.

1.6 Target Users / Audience

1.6 Target Users / Audience

The primary users of ParkStub are drivers who want to find and reserve parking before reaching a destination. The interface is designed for users who need a straightforward way to browse places, check slot status and complete a booking.

A second group is parking operators or administrators who may use the underlying data model to maintain parking places, slot information and reservation records. The current project focuses on the user-facing reservation flow, while more advanced administrative controls can be added later.

The system can also be useful as an academic demonstration for students learning full-stack development. It shows how frontend components, backend routes and database tables cooperate to implement a real-world business process.

1.6 Target Users / Audience

The target audience therefore includes individual drivers, regular parking users, parking-service operators and students or developers studying web application architecture. The design keeps the core actions simple so that a new user can understand the reservation process without technical knowledge.

1.6 Target Users / Audience

User Journey

A typical user begins at the ParkStub home page and browses the listed parking places. After selecting a place, the user can inspect the available slot layout and distinguish available spaces from occupied and reserved spaces. The user then selects a date and an available slot.

The selected details are sent to the backend when the user proceeds with booking. The backend validates the request and records the reservation in MySQL. After the payment flow is completed, the system presents a confirmation view containing the booking information and QR or ticket representation used by the application.

The My Bookings section provides a central place for the user to review reservations. This flow keeps the user experience connected from discovery through confirmation and later booking management.

1.6 Target Users / Audience

System Benefits

The main benefit of ParkStub is reduced uncertainty. Instead of treating every parking place as identical, the application can show different slot conditions for different locations. This makes the information more meaningful to the user.

Another benefit is centralized booking information. The backend and database provide a single source for reservations, making it easier to retrieve booking history and apply booking-related rules consistently.

The project also provides a foundation for future real-time occupancy. If sensors or operator updates are connected later, the same slot-status concept can be extended so that occupied spaces are updated automatically.

1.6 Target Users / Audience

Functional Summary

ParkStub includes registration and login, parking-place discovery, place details, date selection, slot selection, booking creation, payment flow, confirmation, QR/ticket representation, My Bookings and cancellation/refund handling.

The application is organized so that the frontend is responsible for displaying these operations while the backend is responsible for validation and data processing. This separation improves maintainability and makes it easier to modify individual parts without redesigning the whole system.

1.6 Target Users / Audience

Project Perspective

From an academic perspective, ParkStub demonstrates the use of component-based frontend development, server-side routing, relational data modeling and API-based communication. The project also provides opportunities to practice debugging across multiple layers because a single user action may involve the browser, API route and database.

The final system is therefore both a practical parking application and a demonstration of full-stack engineering principles.

1.6 Target Users / Audience

Chapter Summary

This chapter introduced the ParkStub problem, scope, objectives, importance and target users. The system is intended to provide a structured digital parking reservation experience with location-specific slot status and an end-to-end booking flow.

The next chapter defines the hardware, software and development requirements needed to run and maintain the application.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

ParkStub does not require specialized hardware for normal development. A standard laptop or desktop capable of running a browser, Visual Studio Code, Node.js and MySQL is sufficient.

A multi-core processor and at least 8 GB RAM provide a comfortable development environment. More memory is useful when the developer runs the frontend server, backend server, MySQL service, browser, Postman and code editor simultaneously.

Storage is required for the project source code, node_modules packages, database files and screenshots. A stable internet connection is useful when installing packages, accessing documentation and synchronizing source code.

System Requirement Summary

The required software can be installed independently and started in a predictable order: MySQL service first, backend second and frontend third. The browser then connects to the frontend, while the frontend communicates with the backend through the configured API base URL.

2.1 Hardware Requirements

Development Environment

The client side only requires a device with a modern web browser. Since the interface is delivered through React in the browser, the user does not need the development tools installed.

For testing, the development machine should be able to run two local servers at the same time: the React development server for the frontend and the Node.js/Express server for the backend. MySQL also runs as a local service.

A keyboard, mouse and display are sufficient for normal operation. If the system is later connected to real parking sensors, additional hardware such as microcontrollers, ultrasonic sensors or cameras could be introduced, but these are outside the current software implementation.

System Requirement Summary

The required software can be installed independently and started in a predictable order: MySQL service first, backend second and frontend third. The browser then connects to the frontend, while the frontend communicates with the backend through the configured API base URL.

2.2 Software Requirements

Operating System and Runtime

The project can be developed on Windows, Linux or macOS. The current development workflow uses a Windows environment with Visual Studio Code, Node.js, npm and MySQL.

Node.js provides the runtime for the backend and npm manages the backend and frontend JavaScript dependencies. Express is used on the server to define API routes and process HTTP requests.

A modern browser such as Chrome, Edge or Firefox is required to run the React frontend. Postman can be used to test backend endpoints independently from the user interface.

System Requirement Summary

The required software can be installed independently and started in a predictable order: MySQL service first, backend second and frontend third. The browser then connects to the frontend, while the frontend communicates with the backend through the configured API base URL.

2.2 Software Requirements

Database and API Tools

MySQL is the relational database used by ParkStub. It stores persistent application data and allows the backend to execute structured queries for users, places, slots and bookings.

REST APIs provide the communication boundary between the frontend and backend. The frontend sends HTTP requests and receives JSON responses. This approach keeps the client interface independent from direct database access.

Postman is useful for verifying endpoint behavior, status codes and request payloads before connecting the same endpoints to React.

System Requirement Summary

The required software can be installed independently and started in a predictable order: MySQL service first, backend second and frontend third. The browser then connects to the frontend, while the frontend communicates with the backend through the configured API base URL.

2.3 Development Tools and Frameworks

Frontend Tools

React is used to build the ParkStub frontend as reusable components. Components can represent navigation, parking-place cards, slot displays, booking forms, payment views and booking-history elements.

JavaScript manages interaction and application state. CSS controls the visual layout and helps present slot states clearly. The frontend communicates with the backend using HTTP requests.

Visual Studio Code is used as the main development environment. Browser developer tools are useful for inspecting console errors, network requests and frontend state.

System Requirement Summary

The required software can be installed independently and started in a predictable order: MySQL service first, backend second and frontend third. The browser then connects to the frontend, while the frontend communicates with the backend through the configured API base URL.

2.3 Development Tools and Frameworks

Backend and Database Tools

Node.js and Express provide the backend service. The server receives frontend requests, validates input, performs database operations and returns structured responses.

MySQL stores the persistent data. Postman supports API testing, while npm provides dependency management and development scripts.

The combination of React, Node.js, Express and MySQL gives ParkStub a clear full-stack structure that is easy to run locally and suitable for future expansion.

System Requirement Summary

The required software can be installed independently and started in a predictable order: MySQL service first, backend second and frontend third. The browser then connects to the frontend, while the frontend communicates with the backend through the configured API base URL.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design – React Components

ParkStub uses React to build the user interface from reusable components. Parking-place cards can be rendered from data returned by the backend, while slot components can apply different visual states based on whether a slot is available, occupied or reserved.

React state is used for user selections such as the chosen parking place, date and slot. When the user changes a selection, the interface can update without reloading the entire page.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

3.1 Front-End Design – Navigation and User Experience

The frontend contains screens for the home page, parking-place listings, place details, booking, payment, confirmation and My Bookings. Navigation connects these views into one continuous workflow.

Clear status labels are important in the slot view. An available slot can be selected, while occupied and reserved slots are displayed as unavailable so that the user does not accidentally choose them.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

3.1 Front-End Design – Forms and Validation

Registration, login, date selection and booking forms require input validation before data is sent to the backend. Basic validation reduces invalid requests and gives the user immediate feedback.

Final validation remains a backend responsibility because browser-side checks alone cannot protect the database. The server therefore verifies important booking information again.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

3.2 Back-End Design – Node.js and Express

Node.js provides the runtime environment for the ParkStub backend and Express provides routing and middleware support. The backend acts as the central processing layer between the React frontend and MySQL database.

API routes can be organized around resources such as users, parking places, slots and bookings. This makes the server easier to understand and maintain.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

3.2 Back-End Design – REST API Communication

The frontend communicates with the backend using HTTP requests. JSON is used to represent request and response data because it is lightweight and works naturally with JavaScript applications.

The backend returns appropriate response information so that the frontend can display success messages, validation errors or booking details.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

3.2 Back-End Design – Authentication

Authentication provides controlled access to user-specific functions. Registration creates a user record, while login verifies credentials and establishes the user's authenticated session according to the implementation.

Authenticated operations such as viewing personal bookings or performing booking actions can be protected by backend checks.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

3.3 Database Design – MySQL Data Model

MySQL is used because ParkStub contains structured entities and relationships. User records, parking places, slots and bookings have predictable fields and can be related using identifiers.

A normalized relational design helps avoid unnecessary duplication. A booking can reference a user, a parking place and a selected slot instead of copying the entire place or user record into every booking.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

3.3 Database Design – Slot and Booking Records

A slot record represents a physical parking space within a particular place. Its state can be represented as available, occupied or reserved, depending on the application data and booking rules.

Booking records connect users with selected parking places and slots for a chosen date or time. This structure allows the system to retrieve a user's booking history and manage reservation state.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

3.4 Version Control – Project Organization

Version control helps maintain the frontend and backend source code as the application evolves. A Git repository can store changes and provide a history of implementation work.

Separating frontend and backend folders makes the project easier to run because each side has its own package configuration and development command.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

3.5 APIs and Services – Application APIs

ParkStub's primary API boundary is the Node.js/Express backend. The React frontend consumes these APIs rather than directly connecting to MySQL.

Additional external services can be integrated later for real payment processing, maps, notifications or live parking data without changing the basic frontend-backend-database structure.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

3. TECHNOLOGY STACK

Technology Stack Summary – Why This Stack

React provides a component-based interface, Node.js and Express provide a lightweight server, and MySQL provides relational persistence. Together these technologies are widely suited to CRUD applications and transactional workflows.

The stack also supports local development, API testing and incremental feature additions, making it appropriate for a student full-stack project.

Application in ParkStub

In ParkStub, this technology is used as part of the parking reservation workflow. The goal is to keep the user interface responsive while the backend and database remain responsible for persistent application data and business rules.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level Architecture Diagram

The ParkStub architecture follows a three-layer full-stack structure. The React frontend is the presentation layer, the Node.js/Express backend is the application layer, and MySQL is the data layer.

The browser sends user actions to the frontend. When information is required from the server, the frontend sends an HTTP request to an Express API. The backend validates the request, performs the required database operation and returns a JSON response.

4.2 Description of Each Layer

Frontend Layer

The frontend contains the screens and components visible to the user. It is responsible for displaying parking places, slot states, forms, booking details and confirmation information.

The frontend does not directly access MySQL. This separation improves security and keeps database credentials on the server side.

Backend Layer

The backend receives requests, applies business rules and communicates with MySQL. It is responsible for operations such as registration, login, retrieving parking data, creating bookings and handling cancellation-related requests.

4.2 Description of Each Layer

Database Layer

The MySQL database stores persistent information required by the application. The relational structure can represent users, parking places, slots and bookings while preserving relationships between these entities.

When a booking is created, the backend writes the booking information to the database. Later, My Bookings can retrieve the same information for the authenticated user.

Data Flow

A typical data flow is: user selects a place → React requests place/slot information → Express processes the request → MySQL returns records → Express sends JSON → React updates the screen. A booking follows the same pattern with additional validation before the database is changed.

4.3 Deployment Architecture

Local Deployment Architecture

During development, the frontend and backend run as separate local processes. The React development server serves the user interface, while the Express server listens on its configured port. MySQL runs as a database service on the same development machine.

The three components communicate through localhost addresses and configured ports. Keeping them separate makes it easy to restart one layer without rebuilding the others.

For future production deployment, the React application can be hosted on a static frontend platform while the Node.js backend and MySQL database are hosted on appropriate server infrastructure. Environment variables should be used for database credentials and API configuration.

CHAPTER -5 DESIGN

5.1 User Interface Design

The ParkStub interface is designed around the parking reservation journey. The home page introduces the service and allows the user to discover parking places. Place cards provide basic information and lead to a detailed view.

The place-detail screen presents the slot map or slot list. Each slot is associated with a status so that the user can distinguish available, occupied and reserved spaces. The selected slot is visually different from unselected slots.

5.2 ER Diagram / Database Schema

The central entities of the database are User, ParkingPlace, ParkingSlot and Booking. A user can have multiple bookings. A parking place can contain multiple slots. A booking connects a user to a selected place and slot for a reservation.

The database design separates reusable place and slot information from individual booking records. This supports consistent retrieval and reduces duplication.

5.3 Design Principles

The design emphasizes clarity, consistency and separation of concerns. Frontend components manage presentation, backend routes manage application logic and MySQL manages persistent records. This structure makes future modifications easier.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation – Authentication Module

Registration accepts user details and creates an account record. Login checks the supplied credentials and provides access to authenticated features. Authentication is important because bookings and booking history are user-specific.

The frontend provides forms for these operations while the backend validates the incoming data and communicates with MySQL.

6. IMPLEMENTATION

6.1 Module-wise Implementation – Parking Place Module

The parking-place module displays different parking locations. Each location can have its own name, details and slot collection. The frontend renders the locations returned by the backend.

This structure solves the earlier problem of showing identical availability for every place because the slot data is associated with the specific parking location.

6. IMPLEMENTATION

6.1 Module-wise Implementation – Slot Management Module

Each parking place contains multiple slots. The slot view presents three important states: available, occupied and reserved. Only an available slot can be selected for a new reservation.

The backend is the authoritative layer for booking-related state changes. The frontend displays the result returned by the server.

6. IMPLEMENTATION

6.1 Module-wise Implementation – Date and Booking Module

The booking flow collects the selected parking place, date and slot. Before creating a booking, the system checks the request and records the reservation.

Booking information is stored so that the user can later see the reservation in My Bookings.

6. IMPLEMENTATION

6.1 Module-wise Implementation – Payment Module

After selecting the booking details, the user proceeds through the project's payment flow. The payment step is connected to the booking process so that the final confirmation contains the selected reservation information.

The implementation can be extended later with a real payment gateway. The current project demonstrates the application-level payment and confirmation workflow.

6. IMPLEMENTATION

6.1 Module-wise Implementation – Confirmation and QR Module

After successful booking and payment processing, ParkStub presents a confirmation screen. The booking information is organized so the user can verify the place, slot and reservation details.

The QR or ticket representation provides a compact way to represent the booking confirmation within the application's workflow.

6. IMPLEMENTATION

6.2 Front-end Logic – State and Rendering

React state keeps track of selected place, selected date, selected slot and booking information. When a user changes a selection, the relevant component re-renders.

API responses are converted into UI state so that the interface reflects current application data instead of using only hard-coded values.

6. IMPLEMENTATION

6.3 API Integration – Frontend-to-Backend Requests

API integration is the connection point between React and Express. The frontend sends requests to endpoints and processes the JSON responses. Errors are handled by displaying a suitable message or preventing the next booking step.

Testing these endpoints separately in Postman helps identify backend problems before they are combined with the frontend.

6. IMPLEMENTATION

6.4 Authentication & Authorization – User-specific Booking Access

Authentication ensures that a user can access the features intended for an authenticated account. Booking history should be associated with the correct user so that one user does not see another user's records.

Authorization checks can be expanded in the future for administrator functions such as managing places and slots.

6. IMPLEMENTATION

Implementation Summary – End-to-End Flow

The completed implementation connects all major modules: registration/login → parking-place discovery → place details → date selection → slot selection → booking → payment flow → confirmation → My Bookings → cancellation/refund flow.

This end-to-end sequence demonstrates that the application is more than a static frontend: it uses a backend service and persistent database records.

CHAPTER 7- FEATURES

7.1 List of Main Features – Registration and Login

Users can create accounts and log in before using user-specific reservation features. The backend validates account operations and stores user information in MySQL.

Feature Integration

The features are connected through the same React frontend, Express backend and MySQL data flow. A change made by a booking operation can therefore be reflected in the user's later booking history.

7. FEATURES

7.1 List of Main Features – Parking Place Listing

The home or listing view presents available parking places. Each place can have its own information and slot configuration.

Feature Integration

The features are connected through the same React frontend, Express backend and MySQL data flow. A change made by a booking operation can therefore be reflected in the user's later booking history.

7. FEATURES

7.1 List of Main Features – Place Details

Selecting a parking place opens its detailed view. This is where the user can inspect the slots belonging to that particular location rather than seeing a common slot list for every place.

Feature Integration

The features are connected through the same React frontend, Express backend and MySQL data flow. A change made by a booking operation can therefore be reflected in the user's later booking history.

7. FEATURES

7.1 List of Main Features – Available / Occupied / Reserved Slots

Slot status is a central feature. Available slots can be selected; occupied and reserved slots are displayed as unavailable. Different places can contain different status combinations.

Feature Integration

The features are connected through the same React frontend, Express backend and MySQL data flow. A change made by a booking operation can therefore be reflected in the user's later booking history.

7. FEATURES

7.2 How Each Feature Works – Date and Slot Selection

The user selects a date and then chooses an available slot. The selected values are carried forward into the booking request so the backend can create the correct reservation.

Feature Integration

The features are connected through the same React frontend, Express backend and MySQL data flow. A change made by a booking operation can therefore be reflected in the user's later booking history.

7. FEATURES

7.2 How Each Feature Works – Payment and Confirmation

The payment flow follows slot selection and leads to booking confirmation. The confirmation contains the reservation information and the application's QR/ticket representation.

Feature Integration

The features are connected through the same React frontend, Express backend and MySQL data flow. A change made by a booking operation can therefore be reflected in the user's later booking history.

7. FEATURES

7.2 How Each Feature Works – My Bookings

My Bookings retrieves the user's stored reservations and presents them in one place. This allows the user to review previous and current booking information.

Feature Integration

The features are connected through the same React frontend, Express backend and MySQL data flow. A change made by a booking operation can therefore be reflected in the user's later booking history.

7. FEATURES

7.2 How Each Feature Works – Cancellation / Refund

The application includes cancellation and refund-related logic according to the implemented booking rules. This keeps booking management inside the same application instead of requiring a separate process.

Feature Integration

The features are connected through the same React frontend, Express backend and MySQL data flow. A change made by a booking operation can therefore be reflected in the user's later booking history.

CHAPTER -8 TESTING

8.1 Postman Testing – Health Check and Basic API Tests

Postman was used to test the ParkStub backend independently of the React interface. This verifies whether the server returns the expected response before frontend integration is considered.

Testing included valid requests, invalid input and booking-related operations. HTTP status codes and JSON response bodies were checked for consistency.

Testing Result

The testing approach confirms the separation between frontend behavior, backend processing and database persistence. Issues found during development could be isolated by testing each layer separately.

8. TESTING

8.1 Postman Testing – Registration API

The registration endpoint was tested with valid user data and with duplicate or invalid values. Successful requests returned the expected success response, while invalid requests were handled by the backend without creating incorrect records.

Testing Result

The testing approach confirms the separation between frontend behavior, backend processing and database persistence. Issues found during development could be isolated by testing each layer separately.

8. TESTING

8.1 Postman Testing – Login API

Login testing verified that valid credentials can authenticate a user and invalid credentials are rejected. The test confirms that authentication is handled by the backend rather than trusted only to the browser.

Testing Result

The testing approach confirms the separation between frontend behavior, backend processing and database persistence. Issues found during development could be isolated by testing each layer separately.

8. TESTING

8.2 Integration Testing – Parking Place and Slot Retrieval

Integration testing verified that the frontend can retrieve parking-place information and slot data through the backend. The tests also checked that different places can display different slot states.

Testing Result

The testing approach confirms the separation between frontend behavior, backend processing and database persistence. Issues found during development could be isolated by testing each layer separately.

8. TESTING

8.2 Integration Testing – Booking Creation

Booking creation was tested by selecting a place, date and available slot. The backend request was verified and the resulting booking record was checked through the application and database.

Testing Result

The testing approach confirms the separation between frontend behavior, backend processing and database persistence. Issues found during development could be isolated by testing each layer separately.

8. TESTING

8.2 Integration Testing – Payment and Confirmation

The payment and confirmation sequence was tested to ensure that the booking information continues correctly from slot selection through the confirmation screen and QR/ticket representation.

Testing Result

The testing approach confirms the separation between frontend behavior, backend processing and database persistence. Issues found during development could be isolated by testing each layer separately.

8. TESTING

8.3 Tools Used for Testing – Frontend and Backend Testing

Visual Studio Code, browser developer tools and Postman form the main testing toolkit. MySQL tools can be used to inspect stored records and verify that database changes match the expected application behavior.

Testing Result

The testing approach confirms the separation between frontend behavior, backend processing and database persistence. Issues found during development could be isolated by testing each layer separately.

8. TESTING

8.4 Test Cases and Results – Positive and Negative Cases

Test cases included successful registration, failed login, place retrieval, unavailable slot selection, successful booking, duplicate booking conditions, payment flow and booking history. Negative tests are important because the system should not allow invalid or conflicting reservations.

Testing Result

The testing approach confirms the separation between frontend behavior, backend processing and database persistence. Issues found during development could be isolated by testing each layer separately.

8. TESTING

8.4 Test Cases and Results – Booking and Cancellation Verification

The final verification focused on the complete user journey. A successful test is one where the user can move from login through place and slot selection to confirmation and later see the reservation in My Bookings. Cancellation behavior was also checked according to the implemented rules.

Testing Result

The testing approach confirms the separation between frontend behavior, backend processing and database persistence. Issues found during development could be isolated by testing each layer separately.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy – Prepare MySQL

The database service must be running before the backend attempts to access application data. The required ParkStub database and tables should be created according to the project's database setup.

9. DEPLOYMENT

9.1 Steps to Deploy – Install Dependencies

The frontend and backend each have their own package configuration. Running `npm install` in the respective project folders installs the required JavaScript packages when setting up the project on a new machine.

9. DEPLOYMENT

9.1 Steps to Deploy – Start Backend

The backend is started from the backend project directory using the configured development command. The terminal should remain open because it hosts the Express API used by the frontend.

9. DEPLOYMENT

9.1 Steps to Deploy – Start Frontend

The frontend is started from the frontend project directory. The React development server prints a local address that can be opened in the browser.

9. DEPLOYMENT

9.1 Steps to Deploy – Connect Frontend and Backend

The frontend must use the backend's configured API base URL. When the user performs an operation, the request travels from React to Express and the response returns to React as JSON.

9. DEPLOYMENT

9.2 Environment Configuration – API and Database Configuration

Database connection settings and API configuration should be kept in environment variables or project configuration rather than hard-coded credentials. This improves security and makes it easier to move between development and production environments.

9. DEPLOYMENT

9.3 Hosting of Frontend and Backend – Local Development Hosting

The current project can be demonstrated locally with MySQL running as a service, the backend running on its local port and the frontend running on its local development port. This is sufficient for development and faculty demonstration.

9. DEPLOYMENT

9.3 Hosting of Frontend and Backend – Production Hosting Plan

For production, the React frontend can be deployed to a frontend hosting service, the Node.js backend can run on a server platform and MySQL can be hosted on a managed database service. HTTPS, environment variables and restricted database access should be used.

9. DEPLOYMENT

9.3 Hosting of Frontend and Backend – Verification After Deployment

After deployment, the complete flow should be verified: open the frontend, log in, load parking places, inspect place-specific slot status, create a booking, complete the payment flow, view confirmation and check My Bookings. Backend logs and database records can be used to diagnose problems.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development – Frontend and Backend Coordination

A full-stack project can fail even when the frontend and backend work separately if their API paths, ports or response formats do not match. The project therefore required checking the exact request URL and JSON structure used by React.

10. CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development – Port and Server Management

Running two development servers requires separate terminal sessions and correct ports. Keeping the backend terminal running while starting the frontend avoids confusion about whether the API is available.

10. CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development – Database Connectivity

The backend depends on the MySQL service and correct database configuration. Database errors can appear as API failures, so testing the connection independently is an important debugging step.

$$x_0 = x_1 = \dots = x_{n-1} = 1, p_0 = p_1 = \dots = p_{n-1} = 1, q_0 = q_1 = \dots = q_{n-1} = 1, r_0 = r_1 = \dots = r_{n-1} = 1, s_0 = s_1 = \dots = s_{n-1} = 1$$

10.1 Issues Faced During Development – Slot Status Logic

Initially, displaying the same availability for every parking place would not represent realistic parking. The slot data must belong to individual places so that one location can have different occupied, reserved and available spaces from another.

10. CHALLENGES & LIMITATIONS

10.2 Solutions Applied – Place-Specific Slot Data

The slot model was organized around parking-place ownership. This makes the frontend render the slots returned for the selected place rather than applying one global slot pattern to all places.

10. CHALLENGES & LIMITATIONS

10.2 Solutions Applied – Booking Validation

Booking requests should be validated before writing the reservation. In particular, an unavailable slot should not be accepted as a new booking. This protects the database from conflicting reservations.

10. CHALLENGES & LIMITATIONS

10.2 Solutions Applied – API Testing

Postman provides a separate way to test the backend. By verifying endpoints independently, frontend errors and backend errors can be distinguished more easily.

10. CHALLENGES & LIMITATIONS

10.3 Current Limitations or Known Bugs – Current Scope

The present system is a software-based reservation application. It does not automatically read physical occupancy sensors, so live occupancy would need an additional hardware or operator data source.

10. CHALLENGES & LIMITATIONS

10.3 Current Limitations or Known Bugs – Operational Limitations

Production-scale features such as distributed real-time updates, advanced administrator controls, external map integration and a live payment gateway can be added later. These are outside the current demonstrated scope.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features – Real-Time Occupancy

A future version can connect parking sensors or operator updates so that occupied spaces change automatically in the database and frontend.

11. FUTURE ENHANCEMENTS

11.1 Planned Features – Live Slot Synchronization

WebSocket or similar real-time communication can push slot changes to connected users so that availability is updated without manual refresh.

11. FUTURE ENHANCEMENTS

11.1 Planned Features – Maps and Navigation

Map integration can show parking locations, distance and directions from the user's current route.

11. FUTURE ENHANCEMENTS

11.1 Planned Features – Administrator Dashboard

An administrator dashboard can allow operators to add parking places, create slots, update occupancy and review bookings.

11. FUTURE ENHANCEMENTS

11.2 Possible Integrations – Real Payment Gateway

A production version can connect a real payment provider so that transaction status is verified by the server.

11. FUTURE ENHANCEMENTS

11.2 Possible Integrations – Notifications

Email, SMS or push notifications can be added for booking confirmation, reminders, cancellations and changes.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project – Overall System

P
a
r
k
S
t
u
b
i
s
a
f
u
l
l
-
s
t
a
c
k
P
a
r
k
i
n
g

12. CONCLUSION

12.2 What was Achieved – Functional Workflow

T
h
e

p
r
o
j
e
c
t

a
c
h
i
e
v
e
d

t
h
e

m
a
i
n

a
p

12. CONCLUSION

12.2 What was Achieved – Slot Management

A

k

e

y

r

e

q

u

i

r

e

m

e

n

t

w

a

s

t

o

d

i

s

t

i

n

g

12. CONCLUSION

12.2 What was Achieved – Backend and Database

T

h

e

E

x

p

r

e

s

s

b

a

c

k

e

n

d

p

r

o

v

i

d

e

s

t

h

e

12. CONCLUSION

12.3 Skills Learned During Development – Frontend Skills

T
h
e

p
r
o
j
e
c
t

d
e
m
o
n
s
t
r
a
t
e
s

R
e
a
c
t

c

12. CONCLUSION

12.3 Skills Learned During Development – Backend Skills

T

h

e

p

r

o

j

e

c

t

d

e

m

o

n

s

t

r

a

t

e

s

N

o

d

e

.

j

12. CONCLUSION

12.3 Skills Learned During Development – Database Skills

T
h
e

p
r
o
j
e
c
t

d
e
m
o
n
s
t
r
a
t
e
s

r
e
l
a
t
i

12. CONCLUSION

12.3 Skills Learned During Development – Testing and Debugging

T
e
s
t
i
n
g

w
i
t
h

P
o
s
t
m
o
n
i
t
i
n
g

b
r
o
w
s
e
r

d
e

12. CONCLUSION

12.3 Skills Learned During Development – Conclusion

P

a

r

k

S

t

u

b

p

r

o

v

i

d

e

s

a

p

r

a

c

t

i

c

a

l

d

e

CHAPTER -13 REFERENCES

Books and Learning Resources

Pressman, R. S., & Maxim, B. R. Software Engineering: A Practitioner's Approach. McGraw-Hill Education.

Duckett, J. HTML and CSS: Design and Build Websites. Wiley.

Flanagan, D. JavaScript: The Definitive Guide. O'Reilly Media.

Learning resources for React, Node.js, Express and MySQL used during implementation.

CHAPTER -13 REFERENCES

Technical Documentation

React documentation and learning resources for component-based frontend development.

Node.js documentation for the JavaScript runtime environment.

Express documentation for HTTP routing and middleware.

MySQL documentation for relational database operations and SQL.

Postman documentation for API testing.

MDN Web Docs for HTML, CSS, JavaScript and HTTP concepts.

Project Sources

ParkStub source code, database scripts, API tests and project notes maintained by the project team.

CHAPTER -14 APPENDICES

Appendix A – ParkStub Application Screens

This appendix is reserved for screenshots of the ParkStub home page, parking-place listing, place details, slot map, booking page, payment page, confirmation/QR screen and My Bookings screen.

The screenshots should demonstrate that different parking places can show different combinations of available, occupied and reserved slots.

CHAPTER -14 APPENDICES

Appendix B – Sample Code Snippets

Frontend request example:

```
fetch('/api/parking-places').then(res => res.json()).then(data => setPlaces(data));
```

Backend route example:

```
app.get('/api/parking-places', async (req, res) => { /* retrieve places */ });
```

The actual project source should be referenced for the complete implementation.

CHAPTER -14 APPENDICES

Appendix C – Installation / Setup Instructions

1. Install Node.js, npm and MySQL if they are not already installed.
2. Open the ParkStub backend folder in Visual Studio Code and run npm install when setting up on a new machine.
3. Start the MySQL service and make sure the ParkStub database is available.
4. Start the backend with the project's configured development command and keep that terminal running.
5. Open a second terminal, enter the frontend folder and start the React development server.
6. Open the frontend local address shown by the development server in the browser.
7. Test login, parking-place listing, slot status, booking, payment flow, confirmation and My Bookings.

CHAPTER -14 APPENDICES

Appendix D – Geo Tag Photos with Guide

Reserved space for project geo-tag photographs showing the ParkStub project demonstration or presentation with the guide.

Guide: Dr.S.Spandhana

Student 1: Shreenitha.D – 2520030504

Student 2: Srihitha.A – 2520030566

CHAPTER -14 APPENDICES

Appendix E – Review Forms with Guide Signatures

Reserved space for project review forms and guide signatures.

Project: ParkStub – A Full-Stack Parking Reservation and Slot Management System

Guide: Dr.S.Spandhana

END OF REPORT

ParkStub

A Full-Stack Parking Reservation and Slot Management System

Prepared by Shreenitha.D – 2520030504 and Srihitha.A – 2520030566

Under the guidance of Dr.S.Spandhana